Rayls: A Novel Design for CBDCs

Mario Yaksetig[1] and Jiayu Xu[2]

[1] Parfin, Cayman Islands
mario.yaksetig@parfin.io
[2] Oregon State University, USA
xujiay@oregonstate.edu

Abstract. In this work, we introduce Rayls, a new central bank digital currency (CBDC) design that provides privacy, high performance, and regulatory compliance. In our construction, commercial banks each run their own (private) ledger and are connected to an underlying decentralized programmable blockchain via a relayer. We also introduce a novel protocol that allows for efficient anonymous transactions between banks, which we call Enygma. Our design is 'quantum-private' as a quantum adversary is not able to infer any information (i.e., payer, payee, amounts) about the transactions that take place in the network. We achieve high performance in cross-bank settlement via the use of ZK-SNARKs and 'double' batching. Concretely, our transactions consist of a set of commitments and a zero-knowledge proof. As a result, each transaction can pay more than 1 bank at once and, secondly, each of these individual commitments can contain aggregated transfers from multiple users. For example, bank A transfers $1M to a different bank B and that amount is actually a sum of multiple users making transfers to bank B. Commercial banks can then enforce regulatory rules locally within their ledgers. Our system is in production with one of the largest clearing houses in the world and is currently being explored in a CBDC pilot.

Keywords: CBDCs · Privacy · Blockchain

1 Introduction

A Central Bank Digital Currency (CBDC) is a novel form of digital money issued and regulated by national monetary authorities, constituting a sovereign digital representation of fiat currency. Unlike traditional cryptocurrencies, CBDCs are centralized in nature and maintain a one-to-one parity with their physical counterparts, thereby preserving the monetary policy transmission mechanisms. CBDCs can be implemented in two primary models: wholesale (restricted to financial institutions) or retail (accessible to the general public), each presenting distinct implications. The integration of CBDCs into existing financial systems promises enhanced payment efficiency, reduced transaction costs, and improved financial inclusion. This integration, however, typically raises critical questions regarding privacy, cyber security, and the potential side effects on the traditional banking infrastructure.

Presently, many nations are exploring designs for a CBDC. Finding the right balance, however, is a particularly hard problem. Concretely, a CBDC must be sovereign and controlled by a Central Bank (or an equivalent national monetary authority). A CBDC must also be private (at least similar privacy to cash). Simultaneously, this same CBDC must also be auditable to ensure that transactions are compliant with the law. Last, but not least, a CBDC must be performant to allow the citizens of a nation to transact. For example, the most popular commercial bank in India has over 500 million clients. There is no blockchain that can process this type of volume. This is the case for a single commercial bank. This problem gets even harder when at the scale of an entire nation. A possible approach is to use different layer-2 (L2) blockchains (or rollups) to scale an underlying layer-1 (L1) blockchain. This approach, however, comes with many limitations. First, there is a fragmentation of liquidity as traditional L2s each have their own contract and have their funds locked in different 'silos'. Second, there is no privacy if we assume a traditional L2 approach where a sequencer publishes all the transactions that take place in the clear on the underlying L1.

With these limitations mind, and the recent upcoming of central bank digital currencies (CBDCs), how can we expect these currencies to live on a blockchain? Additionally, when designing a currency, a monetary authority must think in decades. Therefore, it is imperative to take into account the possibility of a quantum-computer adversary into the design of such a currency.

1.1 Why not a public blockchain?

Many critics often remark that a CBDC can simply be instantiated on a public blockchain similarly to stablecoins (e.g., USDT and USDC). We describe below why that is a wrong approach.

Scalability Presently, no blockchain can process the number of transactions required to fulfill the needs of a nation transacting. Therefore, the underlying asset cannot simply be a traditional token living on a layer-1 (L1) blockchain.

Privacy It is not reasonable to expect that a CBDC can live on a public blockchain and that all the transactions that take place in a nation are open and available for everyone to see.

Sovereignty From a risk exposure perspective, launching a CBDC as a token on a layer-1 blockchain is not within acceptable parameters for a Central Bank. Concretely, a Central Bank cannot compromise the liveness of its currency to an external validator set of nodes. We highlight that this is a problem and that for periods of time, Ethereum saw elevated censorship rates [1]. Even though these numbers do not illustrate that the network completely censored a specific type of transaction, they show the risk for at least partial censorship, and in many cases a majority, which affects the throughput of that type of transactions. A

Central Bank cannot accept this type of risk. This is one of the fundamental reasons why many banks launch their permissioned blockchain.

1.2 Performance of Existing Privacy Solutions

Existing privacy solutions tend to not perform well for a global setting (i.e., a population of a country). Traditionally, this is either due to the linear cost of producing zero knowledge proofs or due to a large transaction size. Often both. This is the problem that we solve in this work. Concretely, we provide the following (informal) anonymity: 'One of the financial institutions in this subset is making a transfer to at least another one in this set.' We note that, in our setting, a cross-chain transaction can be a financial institution settling balance with another and/or a client of a financial institution transferring funds to another institution., allowing us to provide both a wholesale CBDC and retail CBDC within our design. In effect, these cases are indistinguishable. The explanation for this indistinguishability is simple: both cases involve a debit of a balance and a credit in the same smart contract. The additional complexity of a retail CBDC transaction is 'hidden' inside the privacy ledger, which has a private state.

1.3 Our Contributions

In this work, we introduce a new CBDC solution that:

- is atomic
- is scalable
- is auditable
- is (quantum) private
- unifies liquidity from different institutions
- allows for compliance with existing regulation in a modular manner

We highlight that our design is not exclusive to CBDCs and can be extended, for example, to run on Ethereum layer-1 (L1) and allow for private transactions between L2s (i.e., Validium, Plasma [2]).

2 System Model

2.1 Overview

Our design is simple. First, we assume that the central bank operates a permissioned blockchain[3]. Moreover, each commercial bank runs their own privacy ledger (i.e., private blockchain). We assume that both components run an Ethereum Virtual Machine (EVM) blockchain and that the balances of each commercial bank are private. Each bank has a post-quantum key-pair, which is

[3] Our design can run on any EVM blockchain.

used to obtain shared secrets with all the other banks, which are then used to derive symmetric keys to encrypt transaction information. The core component is each financial institution run a tailored, single-node blockchain interconnected through a central regulatory authority's blockchain (commit chain) that acts as a protocol defining standard messaging service.

2.2 System Architecture

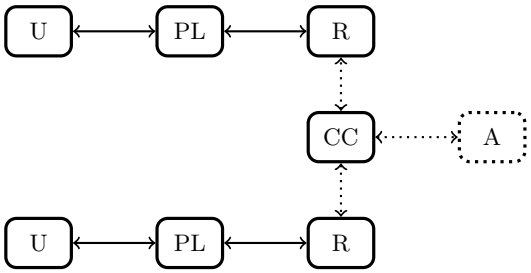

Fig. 1. System Architecture. Users bank with privacy ledgers controlled by the corresponding financial institution. Each privacy ledger uses a relayer to communicate with the commit chain. The commit chain is a layer-1 blockchain that runs smart contracts and also acts a bulletin board. Optionally, we assume the existence of an auditor to monitor the transactions that take place.

In figure 1, we describe the basic architecture of Rayls. A commit Chain (CC) as the central component of the architecture, is a semi-public chain that is being administered by the central bank and can be validated by the public. Each Privacy Ledger (PL) represents an independent financial institution with its own private internal state.

Users (U) of financial institutions are directly connected to the respective Privacy Ledger of corresponding financial institution and have a wallet address on their corresponding privacy ledger.

Each privacy ledger has a wallet address and a balance per currency on the commit chain, representing its total internal balance. Privacy ledgers are connected to the Commit Chain (CC) through Relayers (R), which provide interoperability capabilities between PL and CC.

The Admin role (Central Bank) has special privileges. For instance, a central bank issues a private token on the commit chain and distributes the balance to each PL accordingly. It is through these balances that financial institutions transact between them and the users.

The commit chain guarantees integrity and regulations between privacy ledgers. This prevents double spending and minting more tokens out of thin air through universally verifiable zero-knowledge proofs. Therefore, the central authority can guarantee authenticity of the transactions.

2.3 Assumptions

We assume that the commit chain is a blockchain with a byzantine fault tolerant consensus. This commit chain runs a smart contract for the private transactions while also acting as a bulletin board for parties to publish encrypted information alongside the private transactions. This smart contract enforces a set of predefined rules embedded in the code. For example, a deterministic nullifier is checked when a transaction is submitted and a transaction is discarded if an entity attempts to submit two transactions for a specific time slot. We also assume that each privacy ledger has a quantum-secure key pair to be used for (post-quantum) key agreements in order to establish symmetric keys with each other. We assume the existence of a cryptographically secure hash function to ensure that nullifiers can be securely generated without the risk of collisions.

2.4 Adversarial Model

We assume an adversary $\mathcal{A}$ that wants to subvert the system. To successfully subvert the system, the adversary must be able to perform a double spend transaction, mint funds maliciously and successfully spend such funds, and/or break the privacy of the system. Breaking privacy implies inferring which parties are transacting with each other and/or the amounts of each transaction.

3 Our Design

In this paper, we propose two main approaches. First, the system architecture with the privacy ledgers. We call this Rayls. Second, we propose a new protocol for private transactions inspired in zkLedger [3]. We call this Enygma.

3.1 Formal Specification of User Algorithms

In this section we formally specify the user algorithms in the Enygma protocol. Our description largely follows the terminology in [4, Section 5], except that we add two additional algorithms: SetUp generates the shared secrets between honest parties via an Authenticated Key Exchange (AKE) protocol, and Verify allows parties to check if they have received funds (and if so, how much).

- $\mathsf{SetUp}(1^\lambda)$: Each party i runs $(\mathsf{sk}_\mathsf{i}', \mathsf{pk}_\mathsf{i}') \leftarrow \mathsf{KeyGen}(1^\lambda)$. Each pair of parties i, j runs the AKE protocol Π_{AKE} on inputs $(\mathsf{sk}_\mathsf{i}', \mathsf{pk}_\mathsf{j}')$ and $(\mathsf{sk}_\mathsf{j}', \mathsf{pk}_\mathsf{i}')$, respectively, resulting in shared secrets $s_{i,j} = s_{j,i}$. Party i stores $s_{i,j}$ for all $i \neq j$. Furthermore, each party computes and stores a table of (v, vG) for reasonable values v of funds. There are three possible ways to achieve this:
  - Each party computes the table locally;
  - One party computes and publishes the table, and every other party verifies a fraction of it;
  - Outsource the computation of the table.

- $\mathsf{CreateAddress}(1^\lambda)$: Run $(\mathsf{sk}, \mathsf{pk}) \leftarrow \mathsf{KeyGen}(1^\lambda)$. The output is $(\mathsf{sk}, \mathsf{pk})$.
- $\mathsf{CreateTransferTx}(\mathsf{sk}_i, \mathsf{pk}_j, \mathsf{AnonSet} = \{\mathsf{pk}_1, \ldots, \mathsf{pk}_n\}, b_i, v)$:
  Let pk_i be the public key corresponding to sk_i; if $i, j \in [n]$ (i.e., $\mathsf{pk}_i, \mathsf{pk}_j \in \mathsf{AnonSet}$), proceed as follows:
  1. Obtain the latest block number block.
  2. For all $k \in [n] \setminus \{i\}$ (below we write it as $k \neq i$, i.e., we always assume that $k \in [n]$), calculate random factor $r_k := \mathcal{H}(s_{i,k}, \mathsf{block\#})$. Then define $r_i := -\sum_{k \neq i} r_k$.
  3. Generate commitments
  $$\begin{aligned} C_i &:= (-v)G + r_i H \\ C_j &:= vG + r_j H \\ C_k &:= r_k H \quad \text{for } k \neq i, j \end{aligned}$$
  4. Generate a nullifier to ensure one transaction per bank per block: $\mathsf{nf} := \mathcal{H}(sk_i, \mathsf{block\#})$.
  5. Generate the zero-knowledge proof π. There is a single zero-knowledge proof for the entire transaction. It proves the following:
     - I have enough funds to send in this transaction (i.e., $b_i \geq v$);
     - $(C_1, \ldots, C_n)$ are well-formed (i.e., the random factors $(r_1, \ldots, r_n)$ are well-formed AND I know the secret key sk_i for the slot being debited v amount AND v is credited to at least one of the commitments in this transaction);
     - The nullifier nf is well-formed (i.e., it is the result of a ZK-friendly hash using my secret key sk_i for this round and the block number: $\mathsf{block\#}$).
  6. For each $k \neq i$, calculate $r'_k := \mathcal{H}'(s_{i,k}, \mathsf{block\#})$. This allows each party to check if it has received funds.
  7. Output $\mathsf{tx}_{\mathsf{trans}} := (C_1, \ldots, C_n, \mathsf{nf}, \pi, r'_k)$ (for $k \neq i$).
- $\mathsf{Verify}(\mathsf{sk}_j, \mathsf{tx}_{\mathsf{trans}})$:
  1. Obtain the latest block number block.
  2. For all $i \neq j$, check if there exists an r' in $\mathsf{tx}_{\mathsf{trans}}$ such that $r' = \mathcal{H}'(s_{i,j}, \mathsf{block\#})$. If there is no such i, halt (I am not the receiver of this transaction). Otherwise I know party i has sent funds, and I might be the receiver.
  3. Compute $r_j := \mathcal{H}(s_{i,j}, \mathsf{block\#})$ and $V := C_j - r_j H$. Store $(\mathsf{block\#}, r_j)$ (for the purpose of reading balance).
  4. If $V = 0G$, halt (I am not the receiver of this transaction). Otherwise find entry (v, V) in the pre-computed table, i.e., find v such that $V = vG$.
- $\mathsf{ReadBalance}(\mathsf{sk}_j)$:
  1. Compute corresponding public key pk_j.
  2. Download from smart contract $C := \mathsf{acc}[\mathsf{pk}_j]$.

3. For each (block, r_j) record stored in Verify, subtract $r_j H$ from C. Let B be the final result.
4. Find b such that $B = bG$. Output b.

3.2 Formal Specification of the Smart Contract

The smart contract can process two types of transactions: fund and transfer.

- $\mathsf{Fund}(\mathsf{pk}_i, b)$:
  If $\mathsf{acc}[\mathsf{pk}_i]$ is undefined then set $\mathsf{acc}[\mathsf{pk}_i] := b$, otherwise update $\mathsf{acc}[\mathsf{pk}_i] := \mathsf{acc}[\mathsf{pk}_i] + b$.
- $\mathsf{Transfer}(\mathsf{tx}_{\mathsf{trans}})$ where $\mathsf{tx}_{\mathsf{trans}} = (C_1, \ldots, C_n, \mathsf{nf}, \pi, r')$:
  1. Check if the commitments add up to zero: $C_1 + \cdots + C_n = 0$.
  2. Verify the zero-knowledge proof π.
  3. Tally the commitments: for $i = 1, \ldots, n$, update $\mathsf{acc}[\mathsf{pk}_i] := \mathsf{acc}[\mathsf{pk}_i] + C_i$.

4 Security Analysis

In this section we present the formal security definitions and proofs for the Enygma protocol. Our definitions largely follow [4, Appendix C].

4.1 Security Definitions

We assume an adversary $\mathcal{A}$ that has full visibility over all the transactions that happen in the network and runs a full node of the commit chain. To this end, we first define a basic version of the security game, which is played between a challenger, a smart contract, and the adversary $\mathcal{A}$. At the beginning of the game, the challenger runs $\mathsf{SetUp}(1^\lambda)$, generating the key pair, the shared secrets, and the discrete logarithm table for each party. After that, $\mathcal{A}$ can interact with the challenger and the smart contract in the following manner:

1. $\mathcal{A}$ can instruct the challenger to run any user algorithm in Section 3.1 and see the output (and the resulting transaction, if any, is sent to the smart contract), with the following exceptions:
   - $\mathcal{A}$ cannot instruct the challenger to run SetUp or Verify. (Note that the Verify algorithm requires knowledge of an honest party's secret key.)
   - If the algorithm is $\mathsf{CreateAddress}$, $\mathcal{A}$ only sees the public key pk in the output, and not the secret key sk.
   - If the algorithm is $\mathsf{CreateTransferTx}$, $\mathcal{A}$ specifies the sender's public key pk_i instead of its secret key sk_i.
2. $\mathcal{A}$ can directly send any transaction to the smart contract.
3. $\mathcal{A}$ can instruct the smart contract to process any number of pending transactions and update its state accordingly.

We now define and prove two security properties of the Enygma protocol, namely overdraft-safety and privacy.

Privacy The privacy game is the same as the basic security game, except for the following: at some point, the adversary $\mathcal{A}$ sends two consistent instructions labeled 0 and 1 to the challenger; the challenger samples a bit $b \leftarrow \{0,1\}$ and executes the b-th instruction. At the end of the game, $\mathcal{A}$ outputs a bit b' as a guess for b. $\mathcal{A}$ succeeds if $b' = b$.

Two instructions are *consistent* if they refer to the same user algorithm, and

- If it is CreateTransferTx, then both instructions must have the same AnonSet field; furthermore, if either of the receivers is corrupt, then both instructions must have the same receiver and the same amount.
- If it is ReadBalance, then both instructions must return the same value.

We say a payment mechanism is *private* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ succeeds in the privacy game is upper-bounded by $1/2 + \mathsf{negl}(\lambda)$.

Overdraft-safety. The overdraft-safety game is exactly identical to the basic security game above, and the adversary $\mathcal{A}$ succeeds if at any point a ReadBalance instruction results in a negative value. We say a payment mechanism is *safe against overdrafts* if for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ succeeds in the overdraft-safety game is at upper-bounded by $\mathsf{negl}(\lambda)$.

4.2 Security Proof

Privacy Recall that in the privacy game, the adversary $\mathcal{A}$ must issue two consistent instructions at some point, and guess which one was executed. It is straightforward that two consistent instruction generate identical views unless they are CreateTransferTx; below we consider $\mathcal{A}$ sending two consistent CreateTransferTx instructions.

Let Query be the event that $\mathcal{A}$ queries $\mathcal{H}(s_{i,k}, \mathsf{block})$ for any i, k and block is the block number when $\mathcal{A}$ sends the two consistent instructions. By the security of the AKE protocol, Pr[Query] is negligible. On the other hand, if Query does not occur, then $r_k = \mathcal{H}(s_{i,k}, \mathsf{block})$ is a uniformly random integer in $\mathbb{Z}_p$. with overwhelming probability $r_k = \mathcal{H}(s_{i,k}, \mathsf{block})$ is a uniformly random integer in $\mathbb{Z}_p$; that is, C_i and C_j are Pedersen commitments to $-v$ and v respectively (using independent randomnesses r_i and r_j). By the (perfectly) hiding property of Pedersen commitment, v is independent of C_i and C_j. Since v is not used anywhere other than C_i and C_j in CreateTransferTx, we know that v is independent of $\mathcal{A}$'s view.

Furthermore, for any v, $C_1, \ldots, C_n$ are independent uniformly random group elements, so they leak no information about i (the sender) or j (the receiver). For the remaining parts of $\mathcal{A}$'s view, nf and the r' values leak no information about i or j because $s_{i,k}$ is pseudoranrom in $\mathcal{A}$'s view, and π leaks no information about i or j due to the zero-knowledge property of the ZK scheme.

Overdraft-safety. For any honest party i, let b_i be its balance, i.e., $b_i = \mathsf{ReadBalance}(\mathsf{sk}_i)$. We want to show that $b_i < 0$ with negligible probability at any time.

We first define an ideal protocol as follows: each b_i is initialized to 0, and

- For a $\mathsf{Fund}(\mathsf{pk}_i, v)$ algorithm, set $b_i := b_i + v$;
- For a $\mathsf{CreateTransferTx}(\mathsf{sk}_i, \mathsf{pk}_j, \mathsf{AnonSet}, b_i, v)$ algorithm, set $b_i := b_i - v$ and $b_j := b_j + v$.

Furthermore, only honest $\mathsf{CreateTransferTx}$ algorithms are processed, i.e., all transactions tx^* from the adversary $\mathcal{A}$ are ignored.

Obviously, the probability that $b_i < 0$ at some time is negligible in the ideal protocol, due to the soundness of the zero-knowledge proof (recall that in every transaction, the proof includes the statement $b_i \geq v$). We next show that for any i and at any time, the probability that b_i in the ideal protocol does not match b_i in Enygma is negligible. These two values differ only if $\mathcal{A}$ affects a transaction in one of the following ways:

- *Malicious spending*: $\mathcal{A}$ sends to the smart contract a transaction tx^* where the tuple of commitments $(C_1^*, \ldots, C_n^*)$ is different from all honest transactions, such that the zero-knowledge proof π^* verifies.
- *Double spending*: $\mathcal{A}$ passes to the smart contract an existing honest transaction tx for algorithm $\mathsf{CreateTransferTx}(\mathsf{sk}_i, \mathsf{pk}_j, \mathsf{AnonSet}, b_i, v)$, such that party i's new balance is not $b_i - v$ or party j's new balance is not $b_j + v$ (e.g., $\mathcal{A}$ makes party i send $2v$ amount of funds to party k).

We now show that $\mathcal{A}$ has negligible probability of successfully performing either of these two attacks. For malicious spending, let $\mathcal{A}$'s transaction tx^* be from party i. The zero-knowledge proof π^* verifies only if $(C_1^*, \ldots, C_n^*)$ is well-formed; in particular, this means that the $r_1, \ldots, r_n$ factors within $C_1^*, \ldots, C_n^*$ are all well-formed, and $C_i^* = (-v)G + r_i H$ (because π^* contains a proof of the statement that its sender known sk_i). The only remaining case where $(C_1^*, \ldots, C_n^*)$ is well-formed is that $\mathcal{A}$ sees some honest transaction $(C_1, \ldots, C_n)$ from the same epoch, and replaces $(C_j, C_k) = (vG + r_j H, r_k H)$ with $(r_j H, vG + r_k H)$ (i.e., changes the intended receiver from j to k).[4] However, this attack is infeasible as v is hidden from $\mathcal{A}$ (due to privacy of the protocol).

For double spending, by the definition of Verify, $(C_1, \ldots, C_n)$ implies that party i sends v amount of funds to party j; therefore, the only possible attack without changing $(C_1, \ldots, C_n)$ is that $\mathcal{A}$ copies $(C_1, \ldots, C_n)$ from some honest transaction and sends it more than once. Since we assume that an honest user can only run $\mathsf{CreateTransferTx}$ once per epoch, the block number will change during these two transactions; say they are block and block^*, where block is the block number of the honest transaction (so $\mathcal{A}$ sees $\mathsf{nl} = \mathcal{H}(r_i, \mathsf{block})$). As argued in privacy r_j is uniformly random in $\mathcal{A}$'s view with overwhelming probability, so the probability that $\mathcal{A}$ queries $\mathcal{H}(r_i, \mathsf{block}^*)$ is negligible. But if $\mathcal{A}$ does not make this query, then the correct nullifier $\mathcal{H}(r_i, \mathsf{block}^*)$ is uniformly random in $\mathcal{A}$'s view, so by the soundness of the zero-knowledge proof, the probability that $\mathcal{A}$'s nullifier is well-formed is negligible.

[4] Note that if the commitments are well-formed, all but two of them must be in the form of $r_k H$. In particular, if $\mathcal{A}$ replaces $(C_j, C_k) = (vG + r_j H, r_k H)$ with $((v-1)G + r_j H, G + r_k H)$, then three of the resulting commitments, C_i^*, C_j^*, C_k^*, will not be in that form, so the commitments will not be well-formed.

5 Auditing

We describe the two types of audit that exist in our solution.

Universal Audit. Due to the universal verifiability of the zero-knowledge proofs, anyone can check that the proofs posted in each block are correct. Additionally, an entity that does not wish to verify the ZK proofs of Enygma, can add all the commitments and check if the result adds up to the point in infinity, thus implying that the commitments add up to zero. This addition check is performed at a smart contract level and is transparent and verifiable for anyone observing the system.

Admin Audit. Optionally, under strict auditing requirements, privacy ledgers can share with the admin the post-quantum key-pairs that result in the shared secrets used to derive the encryption keys. This sharing of secret keys allows the admin to open every transaction that takes place in the network, similarly to a 'view' key. We highlight that this key sharing process does not allow the admin to spend funds on behalf of any of the privacy ledgers as this key is only used for encryption.

6 Implementation & Performance

We implemented the complete system using Golang (Go), Solidity, and circom. Concretely, the Privacy Ledger is a fork of Geth (Go implementation of Ethereum) with a custom change of database to use MongoDB instead. This different DB allows this component to provide higher performance to end users. The commit chain is instantiated as an Hyperledger Besu permissioned EVM blockchain. The relayer is a read/write node, also written in Go, that reads events from both the privacy ledger and the commit chain and performs actions accordingly.

We use ZK-friendly cryptographic primitives in the the Enygma smart contract. Concretely, the Baby Jubjub Elliptic Curve as well as the Poseidon [5] hash function. When registering on the commit chain, for the purpose of this implementation, nodes register a CSIDH [6] public-key and perform a post-quantum key agreement with all the other nodes in the network.

Our implementation of a ZK circuit is in circom and covers the entire ZK statement we describe in our protocol description. Concretely, that the sender knows the secret key behind a set of public keys, that the index behind that public key corresponds to a debit, and that the same amount is being credited to other parties involved in the transaction. Additionally, that the commitments of the transaction are well-formed and that the random factors are derived from the pre-established shared secrets.

Our implementation results in 15239 constraints and, as a result, can run on commodity hardware. We provide benchmarks for our implementation on a Mac

mini M1 from 2020 with 16 GB of RAM. A regular Enygma transaction with a k-anonymity of 6 participants takes a total of 4.3 seconds.

Our implementation is currently being tested by a Central Bank in a CBDC pilot and is in production with a clearing house of one of the top 10 biggest nations in the world.

6.1 Balance Rollover

A naive implementation of our design, where every new transaction is automatically tallied, is not viable. This is the case because when a set of commitments is added, the next sender must know how to open their latest hidden balance, which is the previous commitment balance plus new transaction commitment. This is not feasible in a blockchain setting due to the way transactions are processed. To circumvent this issue, we rollover (tally) at the beginning of every new block. Concretely, when the first transaction of the next block is processed, all the previous commitments are tallied. As a result, we have a requirement: privacy ledgers must keep track of the transactions that take place in the network and perform the balance calculation locally before submitting a transaction for the upcoming block(s).

6.2 Optimizations

To have for a greater level of flexibility in our system, we can separate the management (i.e., registration and removal) of banks in the system and enforce a constant-sized anonymity ($k = 6$). This separation allows for the addition and removal of banks without requiring the deployment of a custom new ZK verifier for every update. We have implemented this feature.

Additionally, it is possible to tweak the protocol to use two zero-knowledge proofs instead of one. First, a proof that can contain all of the offline components: the set of commitments that comprise a transaction. Proving the correctness of these elements is the most expensive part of the computation and can be done in advance, as long as the transfer amount is defined ahead of time. The online component, is then simply proving (in zero-knowledge) the ability to open their latest on-chain balance and that they have enough funds for the transfer. This approach results in an offline component of approximately 3 seconds and a real-time component of 1.5 seconds. Resulting in a total of 4.5 seconds. We highlight, however, that this results in an increase of transaction size (two ZK proofs) and transaction cost (verification of two ZK proofs).

7 Extensions

We now describe possible extensions to our protocol.

7.1 E-Cash

Our system can be extended to support an even more private retail CBDC, in this case running inside each privacy ledger. The privacy for each user in the normal case is simply the fact that the operator of the financial institution is expected to keep that data secure and private to external parties. We note, however, that the segregation of privacy ledgers results in a setting where users are directly connected to a financial institution, in this case the commercial bank. This architecture resembles the original e-cash [7] work where users submit a blinded coin and the bank returns a (blindly) signed coin that can be spent privately by the user. As a result, we can use the fact that the privacy ledger effectively produces a chain of blocks to use a trust-minimized e-cash [8] approach, thus allowing users to have cash-like privacy in the transactions that take place.

8 Previous Work

Two projects are most comparable against our work: Zether [4] and zkLedger [3].

Zether uses Σ-Bullets, a combination of Σ-protocols and Bulletproofs over the ElGamal encryption scheme ('in the exponent') to ensure private bi-party transactions where the amount being transferred is private, but the entities involved in the transaction are public. The authors also introduce anonymous Zether, to ensure a bigger anonymity set. The proof size is logarithmic w.r.t the size of the set.

zkLedger uses Pedersen commitments for the transactions along with a 'Sigma' style proof construction. This approach results in linear performance for proof generation, verification time, and proof size. As a result, this approach renders zkLedger impractical for a large number of banks, especially when implementing it to run on a decentralized blockchain.

We use zkLedger as a starting point for our design due to the quantum-secure hiding nature of the Pedersen commitments. Concretely, we explore a setting where we group users into different financial institution buckets and provide a different high-performant ledger for each bucket. We then design a private transaction mechanism based on zkLedger to allow banks to privately transact with each other. This allows for higher-performance as each financial institution can provide fast transactions to their users and banks can transact among each other efficiently.

Our approach follows the rationale that the number of banks is much smaller than the number of users. Therefore, a private payment solution to be used among banks can provide good performance for the system, while also segregating the payments inside the ledger of each financial institution. This is particularly suited for the financial markets due to the inherent separation of client wallets among each institution.

8.1 Differences vs other approaches

Comparison with zkLedger zkLedger uses one ZK proof per commitment in a transaction. Our approach, on the other hand, relies on a single ZK-SNARK for the entire transaction payload. Moreover, we ensure that the random factors in the Pedersen commitments are derived from the (quantum-secure) shared secret between each privacy ledger. This correctness of the random factor is part of the zero-knowledge proof and allows for an auditor to see all the transactions that take place using a view key. zkLedger focuses primarily on cross-bank settlement and resembles our model in that aspect. The original design, however, does not have any programmability or decentralization. We, however, ensure that the validation of transactions takes place on a commit chain that is visible to anyone.

Comparison with (Anonymous) Zether Zether uses discrete-log range proofs for the transactions that take place. Zether does not provide any type of privacy against a quantum computer adversary due to the use of the ElGamal encryption scheme. Zether, however, has one advantage in its ability to read the balance at any point in time just by having the secret key. In our system, users must perform a computation and know of every received transaction to obtain the corresponding random factor from the Pedersen commitment. Zether assumes a monolithic blockchain where every user has an encrypted balance on chain. We diverge away from this model and segregate financial institutions to ensure they can individually provide high performance to their users while having the ability to make efficient 'cross-chain' transfers.

Comparison with L2 Scaling Our architecture is composed of different private blockchains, the privacy ledgers, that can effectively be considered L2's without a public data availability layer. This makes our solution similar to Plasma [2]. A Plasma chain is a separate blockchain anchored to Ethereum main net but executing transactions off-chain with its own mechanism for block validation. Plasma chains use fraud proofs to arbitrate disputes.

9 Discussion

In this section, we cover multiple discussion topics relevant to our work.

9.1 Cross-bank transfers

We expose a mechanism to allow banks to transact with each other. Extending this to transactions between clients in different banks is simple. First, the users lock funds inside their privacy ledgers. Second, the privacy ledger batches these transactions from users into a single Enygma transaction. We note that each Enygma transaction can contain many transactions as a) one transaction can pay more than one bank at once; and b) each of those bank payments can contain funds from different users. This separation is specified in the attached encrypted data.

9.2 Improving the brute-force of the balance

When receiving a transfer, the recipient obtains a commitment in the form of $vG+rH$, where v is the latest balance in the system. Due to the way the random r factor is generated, the recipient is able to obtain its inverse and remove it, thus obtaining vG. This value v, however, is not easily obtainable, due to the nature of the discrete log problem. Therefore, a user must have a mechanism to obtain v from vG. The trivial solution is to perform a brute-force for all the possible balances v. This is traditionally a reasonable approach because the balance of a bank is expected to be within reasonable brute-force ranges. In our protocol description, we propose having a precomputed table to ensure a quick lookup. We discuss three different approaches below:

Precomputation of hidden balances A system entity can precompute a table with all the reasonable values vG for the values v that are in the expected balance range. This list can then be shared with the network and each individual party can perform a random partial check. Ideally, N parties performing a random partial check would catch any malicious values in such a list. This would ensure that the linear balance computation is only performed once.

Ongoing table computation Recipients can compute and store the balance table progressively. This approach results in a variable amount of work per transaction. However, it is possible to, over time, compute the balance table, perform the lookup locally first upon receiving a transaction. If the value is not present, then the recipient can start brute-forcing from the highest value in the database.

Encrypted data. In our setting, since all parties share a secret that they can use to encrypt, the sender can optionally include an encrypted payload containing additional transaction data. The recipient can decrypt the ciphertext and obtain the transfer amount.

9.3 Quantum-Security.

The entire system is at least quantum private in an initial phase due to the use of a quantum-secure key agreement, cryptographically-secure hashing, and zero-knowledge proofs. Therefore, it is possible that the system can rely initially on ZK-SNARKs[9][10][11] and still preserve privacy against a quantum-adversary and, in when in danger of a quantum computer, switch to post-quantum ZK proofs, such as ZK-STARKs [12] or lattice-based [13], thus ensuring end-to-end quantum-security in the system.

The choice of quantum-secure cryptography algorithms for the key exchange is challenging as seen by the recent devastating attacks against isogeny-based cryptograghy [14] and the recent scare against lattice-based cryptography [15]. We remark, however, that failed attempts to break the standardized lattice-based primitives should give more confidence to the community in such primitives. We,

therefore, recommend implementing this system using the Module-Lattice-Based Key-Encapsulation Mechanism Standard[16].

9.4 Trusted Setup

While our scheme is agnostic to the underlying used zero-knowledge (i.e., ZK-SNARK) scheme, we favor Groth16 [9]. We highlight this specific construction due to its lightweight verifier requirements. A limitation of this scheme, however, is the requirement of a trusted setup. While frequently cited as a fundamental problem, we emphasize that this setup can be achieved in a multi-party fashion [17], which can significantly minimize trust assumptions.

10 Conclusion

We proposed a novel central bank digital currency design. We use privacy ledgers, which are a single-node private blockchain, as a segregated and contained silo for each financial institution to run their internal day-to-day operations with high-performance. This resembles the L2 scaling ideas present in Ethereum. We ensure interoperability and connectivity with other financial institutions by introducing the use of a central 'commit chain', resulting in a hub and spoke model. All of the data that flows through this commit chain is either encrypted (i.e., symmetric encryption) using keys derived from a post-quantum key agreement, cryptographically-secure hashes, and in the form of a zero-knowledge proof. This results in extremely strong security and privacy properties. All of our system is EVM-compatible and, even though we have a custom deployment of our system, is easily extendable to existing blockchains (e.g., Ethereum, Polygon, Avalanche). As a result, our proposed Enygma protocol allows for a private, scalable, quantum-ready currency.

Instead of having a global ledger for all users, we separate users into different privacy ledgers. For example, we can group hundreds of millions of users into approximately 400 privacy ledgers[5]. Our Enygma protocol then runs between the privacy ledgers as opposed to between all the users in the system resulting in a linear complexity in number of financial institutions, which is orders of magnitude smaller than the number of actual users in the system.

With this paper, we present our ideas for Rayls and an initial analysis of it, with the hope that our work will stimulate additional fruitful discussion, analysis, and refinements. We are actively testing our design with financial institutions and expect to improve on this design and cover potential gaps in our system.

[5] The number 400 is not arbitrary and reflects a real-world example

References

1. A. SARKAR, "Ethereum ofac compliance dips post-merge upgrade," 2023. [Online]. Available: https://cointelegraph.com/news/ethereum-ofac-compliance-dips-45-post-merge-upgrade
2. J. Poon and V. Buterin, "Plasma: Scalable autonomous smart contracts," 2017. [Online]. Available: https://plasma.io/plasma.pdf
3. N. Narula, W. Vasquez, and M. Virza, "zkledger: privacy-preserving auditing for distributed ledgers," in *Proceedings of the 15th USENIX Conference on Networked Systems Design and Implementation*, ser. NSDI'18. USA: USENIX Association, 2018, p. 65–80.
4. B. Bünz, S. Agrawal, M. Zamani, and D. Boneh, "Zether: Towards privacy in a smart contract world," in *Financial Cryptography and Data Security: 24th International Conference, FC 2020 , Kota Kinabalu, Malaysia, February 10–14, 2020 Revised Selected Papers*. Berlin, Heidelberg: Springer-Verlag, 2020, p. 423–443.
5. L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger, "Poseidon: A new hash function for zero-knowledge proof systems," Cryptology ePrint Archive, Paper 2019/458, 2019. [Online]. Available: https://eprint.iacr.org/2019/458
6. W. Castryck, T. Lange, C. Martindale, L. Panny, and J. Renes, "Csidh: An efficient post-quantum commutative group action," Cryptology ePrint Archive, Paper 2018/383, 2018. [Online]. Available: https://eprint.iacr.org/2018/383
7. D. Chaum, A. Fiat, and M. Naor, "Untraceable electronic cash," in *Advances in Cryptology — CRYPTO' 88*, S. Goldwasser, Ed. New York, NY: Springer New York, 1990, pp. 319–327.
8. M. Yaksetig, "A trust-minimized e-cash for cryptocurrencies," Cryptology ePrint Archive, Paper 2024/444, 2024. [Online]. Available: https://eprint.iacr.org/2024/444
9. J. Groth, "On the size of pairing-based non-interactive arguments," Cryptology ePrint Archive, Paper 2016/260, 2016. [Online]. Available: https://eprint.iacr.org/2016/260
10. S. Ames, C. Hazay, Y. Ishai, and M. Venkitasubramaniam, "Ligero: Lightweight sublinear arguments without a trusted setup," Cryptology ePrint Archive, Paper 2022/1608, 2022. [Online]. Available: https://eprint.iacr.org/2022/1608
11. A. Gabizon, Z. J. Williamson, and O. Ciobotaru, "PLONK: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge," Cryptology ePrint Archive, Paper 2019/953, 2019. [Online]. Available: https://eprint.iacr.org/2019/953
12. E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev, "Scalable, transparent, and post-quantum secure computational integrity," Cryptology ePrint Archive, Paper 2018/046, 2018. [Online]. Available: https://eprint.iacr.org/2018/046
13. V. Lyubashevsky, N. K. Nguyen, and M. Plancon, "Lattice-based zero-knowledge proofs and applications: Shorter, simpler, and more general," Cryptology ePrint Archive, Paper 2022/284, 2022. [Online]. Available: https://eprint.iacr.org/2022/284
14. W. Castryck and T. Decru, "An efficient key recovery attack on sidh," Cryptology ePrint Archive, Paper 2022/975, 2022. [Online]. Available: https://eprint.iacr.org/2022/975
15. Y. Chen, "Quantum algorithms for lattice problems," Cryptology ePrint Archive, Paper 2024/555, 2024. [Online]. Available: https://eprint.iacr.org/2024/555

16. NIST, "Module-lattice-based key-encapsulation mechanism standard," 2024. [Online]. Available: https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.203.pdf
17. V. Nikolaenko, S. Ragsdale, J. Bonneau, and D. Boneh, "Powers-of-tau to the people: Decentralizing setup ceremonies," Cryptology ePrint Archive, Paper 2022/1592, 2022.